Standalone Progressive Retrieval Attention: A PyTorch Architecture for URI-Addressed Memory Expansion

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2, 2026

Abstract

This paper describes a standalone PyTorch research prototype for Progressive Retrieval Attention (PRA). A small decoder-only transformer is augmented with explicit reference handles, an in-memory URI resolver, layer-specific key/value memory, and a cross-attention branch over selected reference memories. We formalize the reference graph, selection and expansion operators, runtime/cache state, lifecycle, complexity, and per-example cache construction. A two-phase evaluation separates ordinary language modeling from learning a new reference path. Preliminary two-seed experiments compare four-layer SelfAttention, four-layer PRAttention, and a two-plus-two hybrid on WikiText-2. At a matched 524,288-token pretraining budget, mean test losses are 4.7691, 4.7457, and 4.7574, respectively. After reference-conditioned fine-tuning, valid references improve loss relative to disabled references by 0.1165 for full PRA and 0.0841 for the hybrid; shuffled-reference penalties of 0.0145 and 0.0122 provide weaker but consistent evidence of content sensitivity. Freezing the base model preserves plain-text loss exactly for PRA variants, whereas the fully fine-tuned SelfAttention control increases plain-text loss by 0.1859. These results are preliminary engineering evidence, not a claim of general superiority: the study uses two seeds, one small model scale, a generated reference task, and same-width rather than parameter-matched controls. We specify the five-seed, parameter-matched, counterfactual, systems-diagnostic matrix required for a publication-grade follow-up without fabricating its results.

1 Introduction

Progressive Retrieval Attention (PRA) is motivated by a common mismatch in long-context language modeling. Many tasks require access to large external context, but the relevant evidence is sparse, hierarchical, and often discovered only after partial reasoning. A conventional decoder-only transformer receives a flat token sequence. Retrieval-augmented generation improves this interface by prepending retrieved passages, but the retrieved text still becomes ordinary prompt context [6]. PRA instead represents external context through lightweight reference handles that can be resolved into URI-addressed memory and attended to through a dedicated memory path.

This paper documents the first standalone PyTorch prototype in the PRA program. The prototype is not a production long-context model. It is a controlled experimental instrument. Its goals are to make the model definition inspectable, expose reference use through explicit metadata, and support ablations that would be difficult to interpret in a large pretrained model. The contribution now includes an initial empirical pilot and an evaluation design intended to make failure visible rather than merely demonstrate that a memory branch can lower loss.

The implementation consists of four main components. First, a tiny GPT-style language model provides causal decoder blocks. Second, a reference system parses tokens such as `<REF_1>` and

maps them to URI-addressed references. Third, a resolver and cache encode referenced documents separately, storing per-layer key/value tensors. Fourth, a PRA attention layer retrieves cache entries by summary-vector similarity and adds a scaled cross-attention memory branch to ordinary causal self-attention.

2 Model Definition

2.1 Decoder Backbone

Let $x_{1:T}$ be a token sequence. The model embeds tokens and absolute positions:

$$H^{(0)} = E_{\text{tok}}(x_{1:T}) + E_{\text{pos}}(1:T). \tag{1}$$

For each layer $\ell \in \{0, \dots, L-1\}$, the implemented block is a pre-normalized residual decoder block:

$$\tilde{H}^{(\ell)} = \text{LN}_1(H^{(\ell)}), \tag{2}$$
$$A^{(\ell)} = \text{PRAttn}_\ell(\tilde{H}^{(\ell)}), \tag{3}$$
$$U^{(\ell)} = H^{(\ell)} + A^{(\ell)}, \tag{4}$$
$$H^{(\ell+1)} = U^{(\ell)} + \text{FFN}_\ell(\text{LN}_2(U^{(\ell)})). \tag{5}$$

The feed-forward network is a two-layer MLP with GELU activation and hidden width $4d$. The output head maps the final normalized hidden state to vocabulary logits. This matches the implementation in `TinyPRAModel` and `PRATransformerBlock`.

2.2 Causal Self-Attention

For hidden states $X \in \mathbb{R}^{B \times T \times d}$, the local branch computes multi-head causal self-attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \tag{6}$$
$$O_{\text{local}} = W_O \,\text{merge}\left[\text{softmax}\left(\frac{QK^\top + M_{\text{causal}}}{\sqrt{d_h}}\right)V\right]. \tag{7}$$

The implementation stores a lower-triangular causal mask with maximum sequence length configured by `PRAConfig.max_seq_len`.

3 Reference Graph and Runtime State

3.1 Reference Graph

Let the external context be a directed attributed graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \tag{8}$$

where each node $v \in \mathcal{V}$ is a tuple (u, z, s, m) containing a stable URI u, content z, summary s, and metadata m. An edge $(v_i, a, v_j) \in \mathcal{E}$ states that anchor a in v_i identifies or expands to v_j. The current implementation stores these relations through reference handles, URI fragments,

and optional children exposed by the resolver. The graph formalism is broader than the current one-level WikiText pilot and makes recursive expansion explicit.

For input sequence x, the reference table exposes a candidate set $\mathcal{H}(x) \subseteq \mathcal{V}$. A resolver

$$\rho(u, p, t) \to (z, s, \text{children}, m) \tag{9}$$

maps URI u, authorization policy p, and logical time or version t to a resolved object. Including p and t in the interface clarifies two systems requirements that the in-memory prototype currently simplifies: access control and freshness.

3.2 Selection and Expansion Operators

At layer ℓ, a selection operator ranks candidate summaries using query $q_{t,\ell}$:

$$\mathcal{S}_{k,\tau}(q_{t,\ell}, \mathcal{C}_t) = \text{TopK}_u \left\{ \cos(q_{t,\ell}, s_u) \mid \cos(q_{t,\ell}, s_u) \geq \tau \right\}. \tag{10}$$

The prototype sets $q_{t,\ell}$ to the last hidden state entering the layer, uses mean token embeddings for s_u, and retrieves at most k entries. An expansion operator

$$\mathcal{X}(A, \mathcal{G}, b, d) \to A' \tag{11}$$

maps active set A to an expanded set A' under branching budget b and depth budget d. The resolver supports child anchors, but the pilot does not yet learn or benchmark $\mathcal{X}$; it is included here to define the intended systems boundary.

3.3 Cache and Lifecycle State

The runtime state before token t is

$$\Omega_t = (\mathcal{G}, \mathcal{H}_t, A_t, \mathcal{C}_t, p_t, \nu_t), \tag{12}$$

where A_t is the active reference set, $\mathcal{C}_t$ the resident encoded cache, p_t policy state, and ν_t provenance/version state. A cache entry is keyed conceptually by $(u, v, \ell, m, \theta, p)$: URI, content version, layer, model identity, relevant encoder parameters, and policy domain. The prototype key is only URI because each cache is rebuilt per example; persistent or shared caches require the fuller identity.

One runtime transition can be written

$$A_t = \mathcal{S}_{k,\tau}(q_t, \mathcal{C}_t), \tag{13}$$

$$A'_t = \mathcal{X}(A_t, \mathcal{G}, b, d), \tag{14}$$

$$\mathcal{C}'_t = \text{admit}(\text{encode}(\rho(A'_t))), \tag{15}$$

$$(y_t, \Omega_{t+1}) = \text{attend}(x_{\leq t}, \mathcal{C}'_t, \Omega_t). \tag{16}$$

In the pilot, resolution and encoding happen before each example forward pass, $\mathcal{X}$ is one-level identity, admission retains all provided references, and the cache is discarded when the next example begins. This explicit specialization prevents claims about recursion, persistence, or learned admission that the code does not yet support.

4 Reference Handles and Resolver

4.1 Reference Tokens

The prototype uses explicit textual handles of the form `<REF_n>`. A regular expression parser extracts occurrences and optionally attaches entries from a `ReferenceTable`. Each reference table entry stores an id, token, URI, summary, and metadata. Legacy `!!ref:uri!!` syntax is retained only for compatibility.

4.2 URI and Anchor Semantics

References point to simple URIs such as `mem://doc1`. Anchored locations use URI fragments, for example `mem://doc1#section.a`. Utility functions split the URI base from the anchor, compute parent anchors, and construct child anchors. The current in-memory resolver stores a dictionary from URI to text plus an optional dictionary from URI to summary. If a URI is requested with `summary_only=True`, the resolver returns the summary in place of the full text. Child references are exposed when stored keys extend the requested URI with hierarchical suffixes.

4.3 Resolved References

Given a handle r with URI u, the resolver returns

$$\text{resolve}(u) = (\text{text}_u, \text{summary}_u, \text{children}_u, \text{metadata}_u). \tag{17}$$

This simple resolver is sufficient for synthetic and local experiments. Future systems can replace it with file, database, vector-index, search, or web resolvers while preserving the same high-level interface.

5 Layer-Specific Memory Cache

5.1 Cache Entries

The PRA cache stores entries of the form

$$C_u = (u,\ \text{text}_u,\ \text{summary}_u,\ s_u,\ \{(K_{u,\ell}, V_{u,\ell})\}_\ell). \tag{18}$$

Here $s_u \in \mathbb{R}^d$ is a summary vector used for retrieval, and $(K_{u,\ell}, V_{u,\ell})$ are detached key/value tensors projected for layer ℓ. The implementation exposes an abstract `PRAMemoryCache` and a dictionary-backed `PRASimpleMemoryCache`.

5.2 Reference Encoding Pass

Before a batch forward pass, the training code builds a cache from batch metadata. For each referenced document:

1. Resolve the URI to text and summary.
2. Tokenize the reference text and truncate to `max_seq_len`.
3. Tokenize the summary and compute a prototype summary vector by averaging summary-token embeddings.

4. Run the reference text through the same model path with PRA memory disabled.

5. At each layer, take the pre-attention normalized hidden state and project it through that layer's W_K and W_V.

6. Store the resulting detached layer-specific tensors in the cache.

This design is deliberately conservative. It avoids recursive PRA during first-level cache construction and ensures that external memory for layer ℓ uses the same projection matrices as the layer that will consume it.

6 PRA Attention

6.1 Summary-Based Retrieval

At generation or training time, if the cache is non-empty, the current implementation uses the last hidden state before attention as a retrieval query. For a query $q \in \mathbb{R}^d$ and cache summary vectors s_u, retrieval scores are cosine similarities:

$$\text{score}(u \mid q) = \frac{q^\top s_u}{\|q\|_2 \|s_u\|_2}. \tag{19}$$

The cache returns the top-k entries. Entries with similarity below `trigger_threshold` are ignored. Library defaults are `top_k_refs=2` and `trigger_threshold=0.2`. The reference pilot deliberately uses `top_k_refs=5`, `trigger_threshold=-1.0`, and `memory_alpha=1.0` so that all one-to-five candidate references are available and the experiment tests the memory path before introducing a selective trigger. This is an experimental simplification, not the intended final retrieval policy.

6.2 Memory Cross-Attention Branch

For selected URI set $\mathcal{U}$ and layer ℓ, the implementation concatenates cached keys and values across the selected entries:

$$K_{\mathcal{U},\ell} = \text{concat}_{u \in \mathcal{U}} K_{u,\ell}, \quad V_{\mathcal{U},\ell} = \text{concat}_{u \in \mathcal{U}} V_{u,\ell}. \tag{20}$$

It then computes a memory branch using the same query tensor Q from the local branch:

$$O_{\text{mem}} = W_{O,\text{mem}} \text{merge}\left[\text{softmax}\left(\frac{QK_{\mathcal{U},\ell}^\top}{\sqrt{d_h}}\right) V_{\mathcal{U},\ell}\right]. \tag{21}$$

The returned attention output is

$$O = O_{\text{local}} + \alpha O_{\text{mem}}, \tag{22}$$

where α is `memory_alpha`, defaulting to 0.5. If the cache is empty, PRA is disabled, no selected entry passes the threshold, or the selected entry lacks layer-ℓ memory, the layer returns only the local self-attention output.

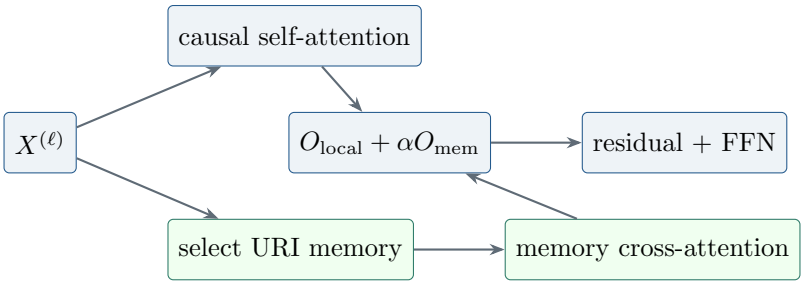

Figure 1: Implemented PRA block. Local causal attention remains intact; selected layer-specific memory contributes through a separately projected residual branch.

6.3 Cross-Attention Rather than Native KV Injection

The standalone prototype uses a separate cross-attention memory branch. Direct concatenation into native self-attention KV is listed as an ablation and future integration path, but it is intentionally not the default mechanism. Cross-attention cleanly separates local causal attention from external memory attention and avoids immediate complications from rotary position embeddings, grouped-query attention, and production cache layout. This makes the prototype suitable for controlled research before adapting large pretrained models.

7 Complexity and Resource Model

Let L be the number of decoder layers, L_p the number of PRA-enabled layers, T local sequence length, r the number of resolved references, m_i the token length of reference i, $M = \sum_{i \in A} m_i$ the selected resident memory length, d hidden width, and w bytes per cached scalar. Ignoring heads and constant factors, local decoder attention costs

$$O(LT^2 d). \tag{23}$$

The prototype separately encodes every resolved reference through the model, costing approximately

$$O\left(Ld \sum_{i=1}^{r} m_i^2\right) + O\left(L_p d^2 \sum_{i=1}^{r} m_i\right) \tag{24}$$

for reference self-attention and layer-specific K/V projection. Memory cross-attention adds

$$O(L_p T M d) \tag{25}$$

to a full-sequence forward pass. The resident K/V payload is approximately

$$B_{\text{cache}} = 2wL_p M d \tag{26}$$

bytes, excluding summaries, object metadata, allocator overhead, and duplicated entries. Each PRA layer adds one $d \times d$ memory output projection plus bias in the current implementation, or $d^2 + d$ parameters. Full PRA therefore adds $L_p(d^2 + d)$ parameters relative to the corresponding custom local block.

These costs explain why shorter visible prompts alone are not a sufficient efficiency result. If every reference is encoded once per example and never reused, $\sum_i m_i^2$ can dominate. PRA becomes

attractive only when selection keeps M small, encoding is amortized across tokens or requests, summaries avoid unnecessary resolution, or quality improves under a fixed compute budget. A systems evaluation must decompose resolver time, tokenization, reference encoding, selection, memory attention, local model compute, cache transfer, and synchronization.

The current per-example cache adapter prevents semantic leakage but serializes examples within a minibatch. A production design would pack memory tensors and associate each sequence with offsets or masks. Its complexity can approach the same aggregate arithmetic while recovering parallel execution, provided selection and cache identity remain isolated.

8 Dataset and DataLoader Pipeline

8.1 Dataset Schema

Datasets are represented by three JSONL files per stage:

- `documents.jsonl`: URI-addressed documents, summaries, titles, and anchors.
- `references.jsonl`: reference ids, URIs, summaries, anchors, and metadata.
- `questions.jsonl`: prompts, answers, expected reference ids, and expected anchors.

The loaded sample type is `QuestionSample`, containing a question, answer, list of `ReferenceSample` objects, target reference ids, and metadata. The current registry includes synthetic memory QA, hierarchical synthetic references, code repositories, Wikipedia, books, technical documentation, and GitHub-style repositories.

8.2 Collation

The `PRACollator` tokenizes each question and answer, forms next-token labels, pads variable-length rows, builds a per-sample reference table, and preserves non-tensor metadata. Prompt labels are replaced by the padding/ignore id, so reference-conditioned loss is computed only over answer-continuation tokens. This metadata is essential: the training step uses it to resolve and encode references before the main model forward pass.

8.3 Data Module

The `PRADataModule` used by synthetic and repository-style stages builds a compact tokenizer from questions, answers, summaries, and reference texts. The WikiText-2 study instead trains a 2,000-token BPE tokenizer with whitespace pre-tokenization and reserves `<REF_1>` through `<REF_5>`. The serialized phase-1 tokenizer is restored for phase 2 and for post-fine-tuning plain-text evaluation. This prevents vocabulary drift from masquerading as an architectural effect.

8.4 Reference-Conditioned WikiText-2

The pilot generator retains natural WikiText language and creates *reference-conditioned continuation*, not conventional question answering. Each eligible source entry is divided into one to five earlier parts plus a tail. Earlier parts become URI-addressed references; the final 8–16 words of the tail become the prediction target. The visible prompt contains only reference handles and the neutral instruction `Continue the referenced WikiText passage:`. It therefore does not provide a local lexical prefix from which the target can be copied. Provenance records include the source

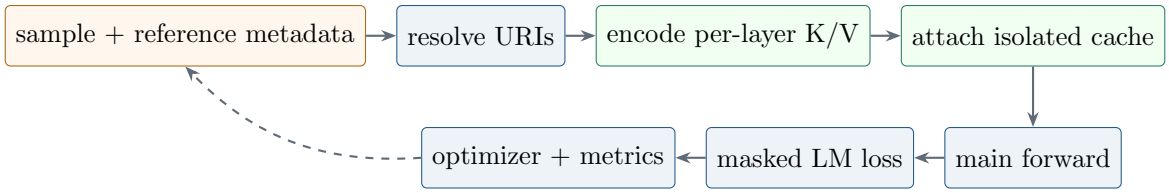

Figure 2: Implemented training/runtime pipeline. Cache construction occurs per example before the main forward pass; prompt labels are ignored in reference-conditioned loss.

entry, part boundaries and identifiers, candidate and intended references, tail span, token distance, part count, local-context sufficiency, and relation type.

The current generator labels all earlier parts as intended references and marks local context as insufficient. This gives a useful first stress test but does not identify a single causal evidence span. Future datasets should distinguish necessary from merely available parts and stratify examples by semantic dependency and reference distance.

9 Training Objective and System

The training objective is standard autoregressive cross-entropy:

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_{\le t}, C), \tag{27}$$

where C is the batch-specific PRA cache constructed from metadata. The training loop uses AdamW, optional learning-rate warmup, gradient accumulation, gradient clipping, optional mixed precision, checkpointing, early stopping, and TensorBoard/W&B/ClearML-compatible logging hooks.

The implemented PRA batch step is:

1. Move tensor fields to the target device.
2. For each example, build a fresh PRA cache from only that example's metadata.
3. Attach that cache to the model and all PRA attention blocks.
4. Run a one-example forward pass with PRA memory enabled and concatenate the resulting logits across the minibatch.
5. Compute cross-entropy with padding ignored.

Per-example cache isolation is semantically important. A previous batch-union implementation allowed each sequence to attend to references belonging to other examples and made within-batch shuffling ineffective. The corrected adapter sacrifices some batching efficiency to prevent cross-example memory leakage. Production batching will require a batched cache representation or an attention mask that preserves this isolation.

9.1 Two-Phase Optimization

Phase 1 trains each architecture as an ordinary causal language model with PRA memory absent. Phase 2 restores the matching phase-1 checkpoint and tokenizer. For full and hybrid PRA, all base parameters are frozen; each memory output projection $W_{O,\text{mem}}$ is zero-initialized and is the only trainable module. Zero initialization makes the initial reference branch an exact no-op and therefore preserves the pretrained function before the first update. The SelfAttention control has no new reference branch, so all of its parameters are fine-tuned on the changed data format. This asymmetry is intentional in the preservation study, but it also means phase-2 reference-task scores compare adaptation strategies as well as architectures. A future joint-tuning study must add parameter-budget-matched controls.

9.2 Execution-Time Accounting

The generic training engine records batch, epoch, and complete-run timing. CUDA is synchronized immediately before and after timed GPU regions so queued kernels are included. Reports contain training, validation, test, and wall-clock durations; processed tokens; sequences seen; optimizer steps; optimizer steps per second; and training tokens per second. Training duration sums synchronized batch regions, validation duration sums explicit validation passes, and wall-clock duration includes orchestration and test evaluation. These definitions should not be interchanged when comparing runtime cost.

10 Evaluation Metrics

The evaluation code reports both language-model and reference-behavior metrics:

- **Loss and perplexity**: autoregressive language-model quality.
- **Answer accuracy**: exact containment of the expected answer in generated output or decoded continuation.
- **Reference retrieval accuracy**: whether expected reference URIs appear in the selected cache.
- **Expected anchor hit**: whether expected anchors appear in cached text or selected URI strings.
- **Expanded reference count**: average number of expanded references per example.
- **Average retrieved tokens**: tokenized size of resolved reference text.
- **Full-context token count**: token cost of a full-context baseline.
- **Cache hit ratio**: fraction of examples whose expected references are represented in cache.
- **Latency and throughput**: wall-clock evaluation time, examples per second, and tokens per second.
- **Preservation loss**: plain WikiText-2 loss before and after reference-format fine-tuning.
- **Reference ablation loss**: answer-token loss with valid, disabled, and shuffled reference memory.

These metrics are designed to test the PRA hypothesis directly. A useful PRA model should not merely achieve high answer accuracy; it should do so while selecting the right references and using fewer retrieved tokens than a full-context baseline.

The implemented disabled condition clears the PRA cache and calls the model with `use_pra_memory=False`. The shuffled condition keeps target tokens and visible prompt structure fixed but replaces each example's reference set by a deterministic half-dataset rotation before rebuilding its isolated cache. A valid-minus-disabled comparison tests whether the memory branch contributes; valid versus shuffled is stricter because it asks whether the contribution depends on reference content rather than only branch activation or formatting. The SelfAttention control is invariant across these conditions by construction.

10.1 Required Systems Diagnostics

Task loss is insufficient to explain a memory system. Table 1 separates signals already emitted by the prototype from publication-grade diagnostics still required.

Table 1: Systems diagnostics and current implementation status.

Diagnostic	Measurement	Status
Reference selection	top-k hit, MRR, NDCG against relevant URIs	URI hit implemented; ranked metrics pending
Attention behavior	token-by-reference heatmaps, entropy, relevant mass	pending instrumentation
Layer utilization	selected entries, memory norm, gate/branch ratio per layer	pending instrumentation
Cache reuse	hit rate by URI/version and avoided encoding tokens	per-run hit only; persistent reuse pending
Latency	resolve, tokenize, encode, select, attend, synchronize	aggregate train/eval timing implemented; decomposition pending
GPU memory	peak allocated bytes and cache payload bytes	current allocation emitted; peak/decomposition pending
Failure cases	wrong URI, stale content, local insufficiency, poisoning, leakage	traces implemented; labeled taxonomy pending

Attention heatmaps should retain URI and token boundaries rather than concatenate memory into an anonymous axis. Layer utilization should distinguish a selected entry from an entry that receives meaningful attention mass. Cache reuse should count avoided encoding work, not merely dictionary hits. Failure reports should preserve prompt, target, candidate references, selected references, content versions, per-layer behavior, and the counterfactual outputs under disabled and shuffled conditions.

11 Experimental Protocol

The pilot compares three same-width architectures:

1. **SelfAttention**: four PyTorch causal SelfAttention layers.

2. **Full PRA**: four local-plus-memory PRAttention layers.

3. **Hybrid PRA**: two lower SelfAttention layers followed by two PRAttention layers.

All use four layers, four heads, width 256, context length 128, dropout 0.1, and the same 2,000-token BPE vocabulary. Phase 1 uses seeds 1 and 7, batch size 2, learning rate 5×10^{-4}, 2,048 optimizer steps, and exactly 524,288 processed tokens per run. Phase 2 restores the matching checkpoint, uses 512 generated reference examples, 512 steps, and learning rate 2×10^{-4}. The runs execute with Python 3.10.11, PyTorch 2.12.1 with CUDA 12.6, and an NVIDIA GeForce GTX 950M.

The same-width parameter counts are 4,216,320 for SelfAttention, 4,479,488 for full PRA, and 4,347,904 for hybrid PRA. Full and hybrid PRA therefore contain 6.2% and 3.1% more parameters than SelfAttention. In phase 2, the PRA trainable counts are 263,168 and 131,584 because only four or two memory output projections are optimized; the SelfAttention control fine-tunes all 4,216,320 parameters. These differences delimit the claims that can be drawn from the pilot.

Table 2: Current pilot hyperparameters. Phase-2 processed tokens vary slightly with generated example lengths.

Setting	Phase 1: plain WikiText-2	Phase 2: references
Tokenizer	shared BPE, vocabulary 2,000	restored phase-1 BPE
Model	4 layers, 4 heads, width 256	matching phase-1 checkpoint
Context/batch	128 / 2	128 / 2
Optimizer budget	2,048 steps; 524,288 tokens	512 steps; mean 46,301.5 tokens
Learning rate	5×10^{-4}	2×10^{-4}
Dropout	0.1	0.1
References	absent	1–5 parts; top-$k = 5$
Threshold/α	inactive	-1.0 / 1.0
Trainable parameters	all	all SA; memory output only for PRA
Seeds	1, 7	1, 7

An approximately parameter-matched SelfAttention control can retain width 256 and increase each four-layer FFN hidden width from 1,024 to 1,152. This adds 262,656 parameters, within 512 parameters of the full-PRA overhead. A second FFN width of 1,088 adds 131,328 parameters, within 256 of the hybrid overhead. These controls require a configurable FFN width and have not yet been run.

The evaluated ablations are:

- valid per-example references;
- a cleared and explicitly disabled PRA path;
- references deterministically shuffled across examples;
- plain WikiText-2 reevaluation after phase 2.

Summary-only memory, recursive expansion, irrelevant/empty references, native KV injection, learned triggering, distance-stratified analysis, and parameter-matched controls remain required future ablations.

11.1 Publication-Grade Experiment Matrix

The current pilot is the first row of a larger preregistered-style matrix. Table 3 marks required comparisons without assigning unmeasured values.

Table 3: Required experiment matrix. "Pending" means no empirical claim is made in this paper.

Axis	Required comparison	Status
Reproducibility	seeds $\{1, 7, 21, 42, 87\}$ with intervals/effect sizes	seeds 1 and 7 complete
Capacity	same-width plus FFN parameter-matched SA controls	same-width complete; matched pending
Reference causality	valid, disabled, shuffled, irrelevant, empty, oracle	first three complete; remainder pending
Cache budget	entries/tokens/bytes; reuse on/off; eviction policies	pending
Memory strength	α sweep including zero and learned gate	$\alpha = 1$ only
Selection	top-k, threshold, oracle router, learned router	forced top-5 only
Transport	separate cross-attention versus native K/V injection	cross-attention only
Adaptation	frozen branch, LoRA, adapters, joint fine-tuning	frozen output branch plus SA control complete
Placement	full PRA, upper-layer hybrid, layer-count sweep	full and 2+2 hybrid complete
Difficulty	distance, candidate count, local sufficiency, recursion depth	metadata present; grouped analysis pending

The five-seed comparison should use fixed tokenizer and split hashes, equal processed-token budgets, and paired seeds. Cache and selector sweeps should initialize from the same checkpoint. Oracle references isolate memory transport from selection; irrelevant and empty controls distinguish content use from generic branch activation. Native K/V injection must account for positional encoding and causal semantics rather than simply concatenate tensors. LoRA, adapters, and joint fine-tuning should be matched by trainable parameter count or reported as separate adaptation regimes.

12 Preliminary Results

Tables 4 and 5 report population means and standard deviations over seeds 1 and 7. They are pilot measurements from the checked-in implementation and report artifacts, not mock values.

All three architectures learn an ordinary token-prediction function and finish within 0.024 test-loss units of one another. This supports the narrow claim that a PRA block behaves comparably to the local baseline when no external memory is present at this scale and budget. It does not establish equivalence under scaling, and the models are not parameter matched.

Disabling reference memory increases mean loss by 0.1165 for full PRA and 0.0841 for hybrid PRA. Shuffling increases loss by 0.0145 and 0.0122. The larger disabled gap shows that the learned

Table 4: Phase-1 WikiText-2 language modeling at a matched 524,288-token budget.

Architecture	Parameters	Test loss	Perplexity	Token acc.	Train s
SelfAttention	4,216,320	$4.7691 \pm .0036$	$117.82 \pm .42$	$.1264 \pm .0006$	94.07 ± 1.43
Full PRA	4,479,488	$4.7457 \pm .0011$	$115.09 \pm .13$	$.1250 \pm .0012$	90.86 ± 1.13
Hybrid PRA	4,347,904	$4.7574 \pm .0017$	$116.44 \pm .19$	$.1271 \pm .0004$	$95.66 \pm .04$

Table 5: Phase-2 reference-conditioned answer-token loss and plain-text retention. Lower is better.

Architecture	Valid	Disabled	Shuffled	Plain Δ	Train s
SelfAttention	$4.7791 \pm .0734$	$4.7791 \pm .0734$	$4.7791 \pm .0734$	+0.1859	$44.82 \pm .45$
Full PRA	$4.6749 \pm .0607$	$4.7914 \pm .0683$	$4.6894 \pm .0563$	0.0000	45.13 ± 2.83
Hybrid PRA	$4.7111 \pm .0356$	$4.7953 \pm .0284$	$4.7233 \pm .0351$	0.0000	$31.14 \pm .02$

memory branch is useful; the consistent but modest shuffled gap is the stronger evidence that the branch depends on which content is supplied. The latter effect remains small and requires confirmation with more seeds, irrelevant-reference controls, and targets with known necessary evidence.

Plain-text reevaluation after phase 2 exactly reproduces the phase-1 losses for full and hybrid PRA because frozen base parameters and zero-initialized isolated output projections protect the local path. The SelfAttention control's mean plain loss rises from 4.7691 to 4.9550, a 0.1859 increase corresponding to about a 20% perplexity increase. This demonstrates preservation by the chosen training strategy, not an inherent guarantee of PRA: the control updates all parameters while PRA updates only reference-specific projections.

Phase-1 mean synchronized validation times are 1.07–1.14 seconds and mean wall-clock times are 97.0–102.0 seconds. Phase-2 mean synchronized training times are 31.1–45.1 seconds, with 46,301.5 mean processed prompt/answer tokens and 512 optimizer steps. Per-example cache isolation makes these timings honest for the prototype but leaves batching optimization as future systems work.

12.1 Reproducibility and Artifact Scope

Resolved per-seed HTML and JSON reports, plots, timing counters, checkpoints, and a six-experiment aggregate are generated under `out/reports`. The aggregate report computes population mean and standard deviation without discarding individual seed values. The current pilot has two seeds, despite the protocol recommending at least three for a final tiny-model comparison. Dataset hashes, parameter-component accounting, peak allocated memory, and a parameter-matched SelfAttention model should be added before treating the result as publication-grade.

13 Relationship to Prior Work

The prototype builds on the transformer attention architecture [9]. It is related to sparse and long-context attention models such as Longformer, BigBird, Reformer, and Routing Transformer [1, 11, 4, 8], but the central question differs: PRA studies whether external context can remain addressable until needed rather than being placed in a single long sequence.

It also relates to memory-augmented and retrieval-conditioned models. Compressive Transformers maintain compressed memories over earlier activations [7]. Memorizing Transformers retrieve

nearest-neighbor memories [10]. RETRO conditions generation on retrieved chunks from a large database [2]. RAG and REALM connect generation to non-parametric retrieval [6, 3]. PRA differs in its emphasis on explicit handles, URI-addressed references, recursive anchors, and per-layer cache construction.

Finally, the implementation anticipates runtime concerns raised by serving systems such as vLLM's PagedAttention [5]. A mature PRA system would require cache admission, sharing, eviction, batching, and memory safety policies.

14 Safety and Systems Correctness

Reference memory expands the model's attack and failure surface. A malicious or compromised reference can poison summaries, embed adversarial instructions, or exploit trusted URI namespaces. URI resolution must therefore preserve source, content digest, version, authorization decision, and resolver identity. Entries from different trust domains should not be merged into an anonymous K/V pool, and outputs should retain enough provenance to reconstruct which objects were resident.

Freshness is part of correctness. A cache key based only on URI can return stale memory after content or model parameters change. Persistent implementations should bind entries to content version, model/adapter identity, layer, representation format, and policy domain. Revocation must invalidate both object lookup and already encoded resident representations.

Batching creates a confidentiality boundary. The corrected prototype constructs a cache per example after a batch-union design exposed cross-example references. Optimized kernels must preserve equivalent masks and should include adversarial tests proving that logits for one sequence are invariant to another sequence's private cache. Cache reuse across users or tenants requires authorization-aware deduplication.

Recursive expansion can amplify work. Cycles, high branching factors, oversized objects, and repeated faults require depth, token, latency, and byte budgets. Resolvers should validate schemes and anchors, detect cycles, and fail closed when authorization or freshness cannot be established. None of these controls is fully implemented in the standalone in-memory resolver; they define the boundary between this research instrument and a deployable runtime.

15 Limitations

The current prototype and pilot have important limitations. Two seeds are sufficient for an engineering pilot but not for a robust empirical claim. Models are same-width rather than parameter matched, and phase-2 trainable parameter budgets differ by design. The reference dataset labels all earlier parts as relevant and uses a generic continuation prompt; it does not yet provide human-verified causal evidence spans. The small shuffled-reference penalty suggests content sensitivity but leaves open how much performance comes from generic domain memory.

Summary vectors are computed by averaging summary-token embeddings, retrieval uses the last hidden state rather than explicit reference-token positions, and the pilot disables thresholding. Recursive anchor expansion is represented in the resolver interface but is not a learned iterative policy. Per-example cache construction prevents leakage but serializes examples inside a nominal minibatch. Reference encodings are detached, so the cache-building path itself receives no gradient in the present phase-2 setup. Evaluation lacks irrelevant, empty, oracle, all-context, and distance-stratified controls. Finally, exact answer containment is useful for synthetic stages but insufficient for open-ended generation. No broad experimental superiority is claimed.

16 Conclusion

The standalone PyTorch PRA prototype provides a concrete testbed for demand-driven, URI-addressed memory expansion. Its main contribution is an inspectable experimental substrate: explicit reference handles, resolver-backed memory, per-layer K/V cache construction, a cross-attention memory branch, isolated per-example caches, and ablations that distinguish branch activation from content use. The initial WikiText-2 pilot establishes three narrow facts: PRA variants retain ordinary language-model performance when memory is absent; an isolated reference path can improve reference-conditioned loss without changing the frozen local function; and correct reference content is modestly better than shuffled content. Larger, parameter-matched, multi-seed studies with stronger controls are required to determine when these mechanisms improve accuracy-cost tradeoffs.

A Reference-Memory Forward Pass Pseudocode

```
for sample in minibatch:
    cache = empty_cache()
    for handle in sample.references:
        resolved = resolver.resolve(handle.uri)
        h_ref = token_embedding(resolved.text) + positions
        entry = CacheEntry(uri=handle.uri)
        entry.summary = mean(token_embedding(handle.summary))
        for layer in model.layers:
            if layer.has_pra:
                entry.kv[layer.id] = layer.project_reference_kv(h_ref)
            h_ref = layer(h_ref, use_pra_memory=False)
        cache.put(detach(entry))

    model.set_pra_cache(cache)
    logits.append(model(sample.input_ids, use_pra_memory=True))

loss = cross_entropy(concat(logits), labels, ignore_index=PAD)
```

The per-sample loop is intentional: cache identity is part of the example. A future batched implementation can vectorize reference encoding but must preserve an example-to-cache mask.

B Simplified PyTorch PRAttention

The following excerpt omits retrieval thresholds, multiple layers, dropout, and shape utilities while retaining the implemented residual-memory idea.

```
class PRAttention(nn.Module):
    def __init__(self, d_model, n_heads, memory_alpha=1.0):
        super().__init__()
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
```

```
        self.v_proj = nn.Linear(d_model, d_model)
        self.local_out = nn.Linear(d_model, d_model)
        self.memory_out = nn.Linear(d_model, d_model)
        self.memory_alpha = memory_alpha

    def forward(self, x, memory_k=None, memory_v=None):
        q = split_heads(self.q_proj(x), self.n_heads)
        k = split_heads(self.k_proj(x), self.n_heads)
        v = split_heads(self.v_proj(x), self.n_heads)

        local_scores = q @ k.transpose(-2, -1) / sqrt(self.head_dim)
        local_scores = local_scores.masked_fill(causal_mask(x), -inf)
        local = softmax(local_scores, dim=-1) @ v
        local = self.local_out(merge_heads(local))

        if memory_k is None:
            return local

        memory_scores = (
            q @ memory_k.transpose(-2, -1) / sqrt(self.head_dim)
        )
        memory = softmax(memory_scores, dim=-1) @ memory_v
        memory = self.memory_out(merge_heads(memory))
        return local + self.memory_alpha * memory
```

Zero-initializing `memory_out` makes the second term exactly zero at phase-2 initialization. Disabling references either omits memory tensors or bypasses the branch explicitly.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[3] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

[4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[7] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[8] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. In *Transactions of the Association for Computational Linguistics*, 2021.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[10] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[11] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.